Computer Sciences faculties USTHB.
2026 - 2027

Embedded Software Development

Chapter 1 : Introduction to Embedded Systems

COURSE 3 : Embedded I/O: GPIO, Sensors, Actuators and Timing

Teacher : Dr. ABADA Lyes.
E-mail : abada.lyes@gmail.com

What will we learn?

At the end of this course, students should be able to:

- Explain how a microcontroller interacts with physical devices.
- Distinguish between sensors and actuators.
- Explain digital input and digital output.
- Understand GPIO configuration.
- Understand pull-up and pull-down resistors.
- Explain input debouncing.
- Understand basic analog acquisition.
- Distinguish ADC and DAC.
- Explain the role of timers in embedded systems.
- Understand blocking and non-blocking timing.
- Explain the relationship between GPIO, sensors, actuators and software.
- Design a simple embedded control loop.

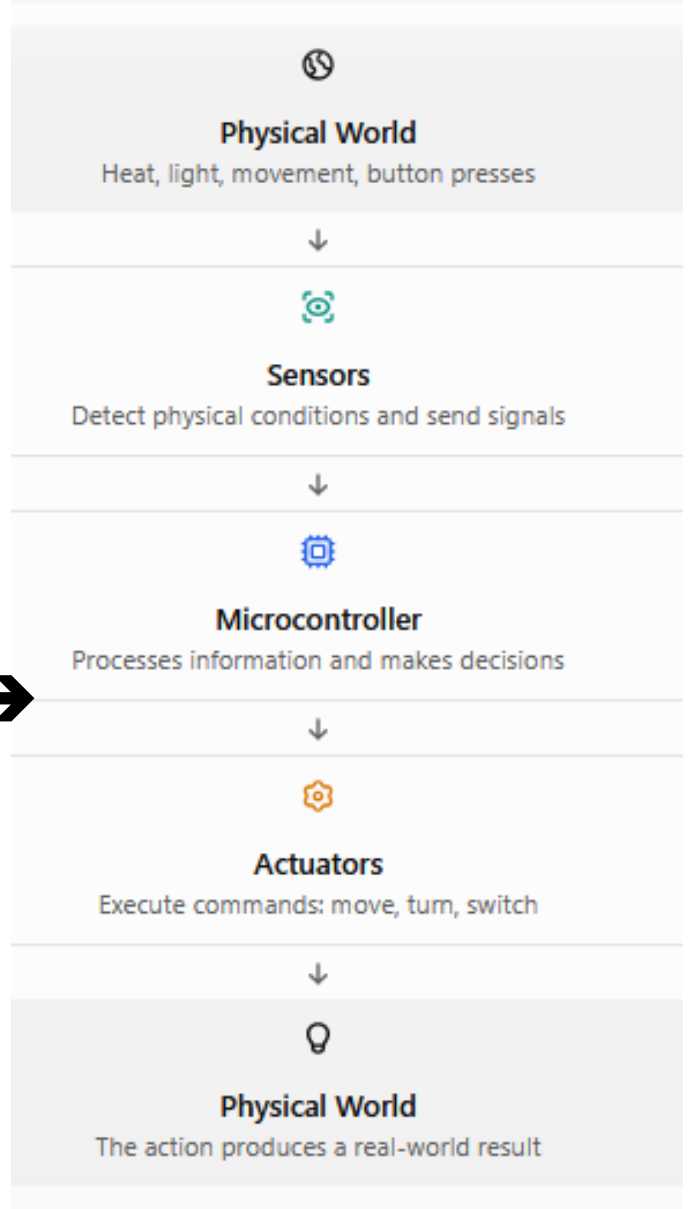
1. FROM CPU TO PHYSICAL WORLD

In Course 2, we studied the CPU, But :

- Can the CPU directly detect that I pressed a button? **No.**
- Can the CPU directly measure the temperature of the room? **No.**
- Can the CPU directly turn a motor? **Not directly.**

➔ **The CPU works with electrical signals and digital data.**

We need interfaces between the processor and the physical world ➔



➔ The sensor allows the system to observe the world.
➔ The actuator allows the system to act on the world.

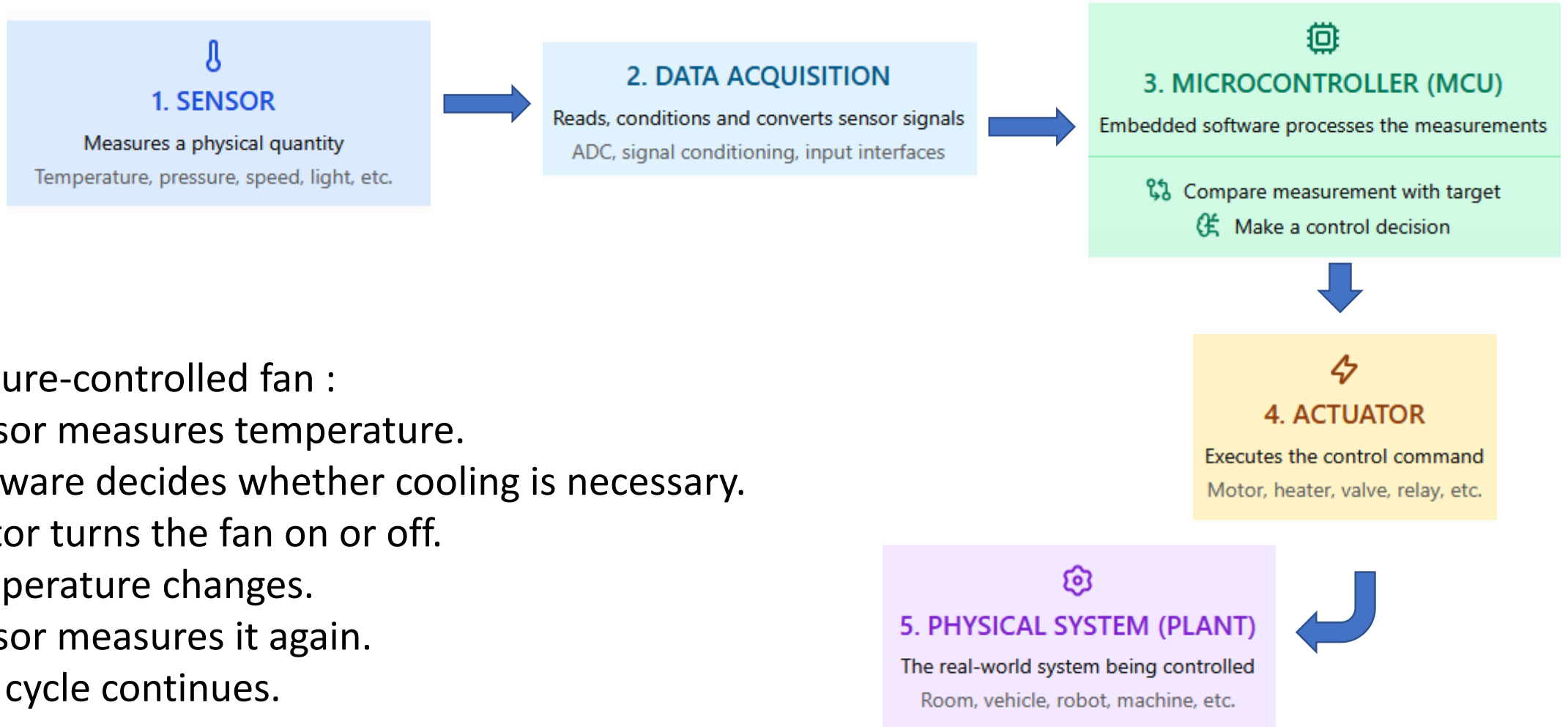
by : Dr. L.ABADA

1. FROM CPU TO PHYSICAL WORLD

Embedded System as a Control Loop

A typical embedded system follows:

This is one of the most important architectures in embedded systems.



Example :

A temperature-controlled fan :

- The sensor measures temperature.
- The software decides whether cooling is necessary.
- The motor turns the fan on or off.
- The temperature changes.
- The sensor measures it again.
- And the cycle continues.

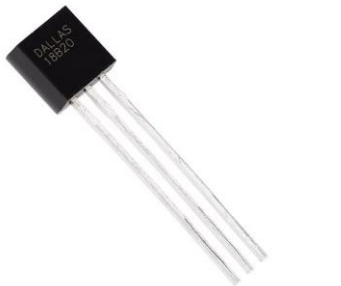
2. SENSORS

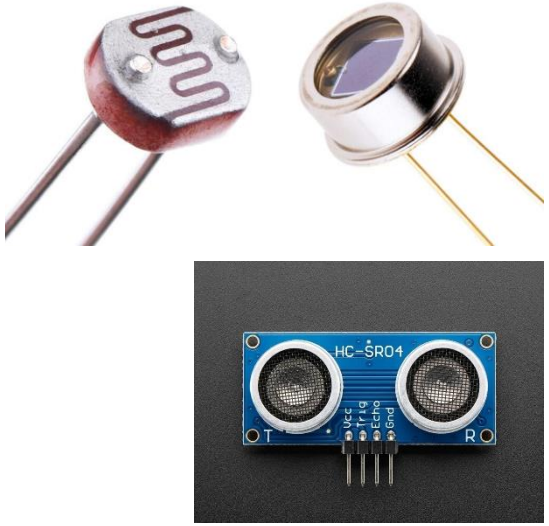
What Is a Sensor?

A **sensor** converts a physical quantity into an electrical signal or digital data.

Examples:

Physical quantity	Sensor
Temperature	Temperature sensor
Light	Photodiode / LDR
Distance	Ultrasonic / ToF sensor
Acceleration	Accelerometer
Pressure	Pressure sensor
Position	Encoder
Magnetic field	Magnetometer



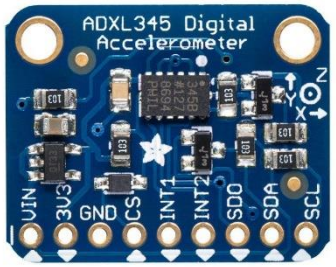




2. SENSORS

Sensor Examples

For example, a drone can contain:



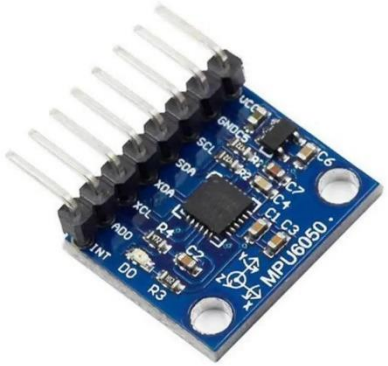
1. Accelerometer

Physical quantity: Acceleration

↓

Output: Acceleration along the X, Y and Z axes

Role: Helps detect changes in movement and the direction of gravity.



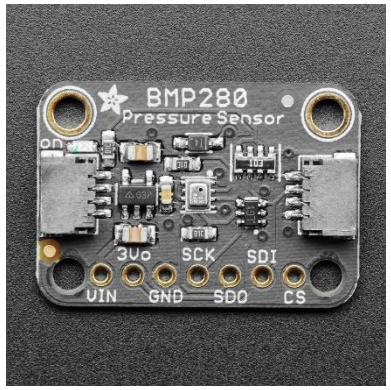
2. Gyroscope

Physical quantity: Angular velocity

↓

Output: Rotation rate around X, Y and Z

Role: Helps the drone maintain its orientation and resist unwanted rotation.



3. Barometer

Physical quantity: Atmospheric pressure

↓

Output: Pressure measurement → estimated altitude change

Role: Helps estimate how high the drone is flying using pressure changes.



4. GPS / GNSS receiver

Physical quantity: Position

↓

Output: Latitude, longitude and estimated altitude

Role: Helps the drone navigate, follow waypoints and return to its launch location.

The microcontroller processes these measurements.

3. ACTUATORS

What Is an Actuator?
An **actuator** converts an electrical command into a physical action.

Examples ➔



1. LED (Light-Emitting Diode)

Electrical command: Turn ON/OFF

↓

Physical output: Light

Example: An LED lights up when a microcontroller activates an output pin.



2. Motor (DC motor)

Electrical command: Apply electrical power

↓

Physical output: Continuous rotation

Example: A DC motor rotates a fan or drives a small robot's wheels.

Microcontroller
Embedded software generates a control command.

↓

Driver / Power Circuit
Provides the voltage and current required by the actuator, when needed.

↓

Actuator
LED · Motor · Servo · Relay · Buzzer · Valve · Solenoid

↓

Physical Action / Output
Light · Rotation · Position · Switching · Sound · Flow · Linear movement



3. Servo motor

Electrical command: Target position

↓

Physical output: Controlled angular position

Example: A servo moves a robot arm to a specified angle. Many hobby servos use PWM control signals.



4. Relay

Electrical command: Activate or deactivate coil

↓

Physical output: Electrical contacts open or close

Example: A microcontroller switches a lamp or pump using a properly rated relay circuit.



5. Buzzer

Electrical command: Apply a suitable signal

↓

Physical output: Sound

Example: A buzzer produces an alarm when a sensor detects a dangerous condition.



6. Valve

Electrical command: Open or close valve

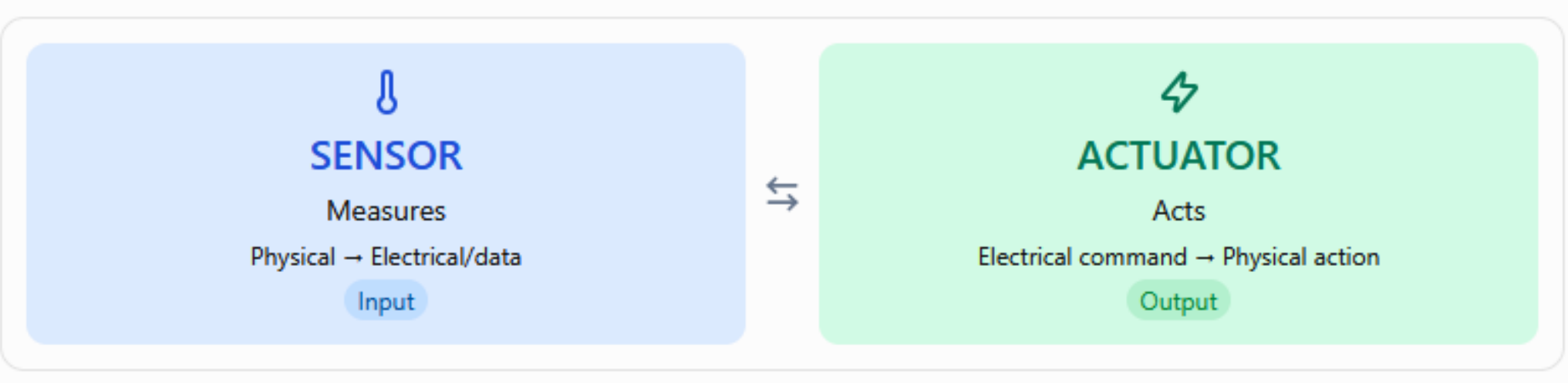
↓

Physical output: Fluid flow is controlled

Example: An electrically operated valve controls water flow in an automatic irrigation system.

Sensor vs Actuator

A sensor measures something in the physical world, while an actuator performs an action in the physical world.



Sensor = Input: detects or measures a physical quantity.

Actuator = Output: converts a control command into an action.

Important : A system can contain both.

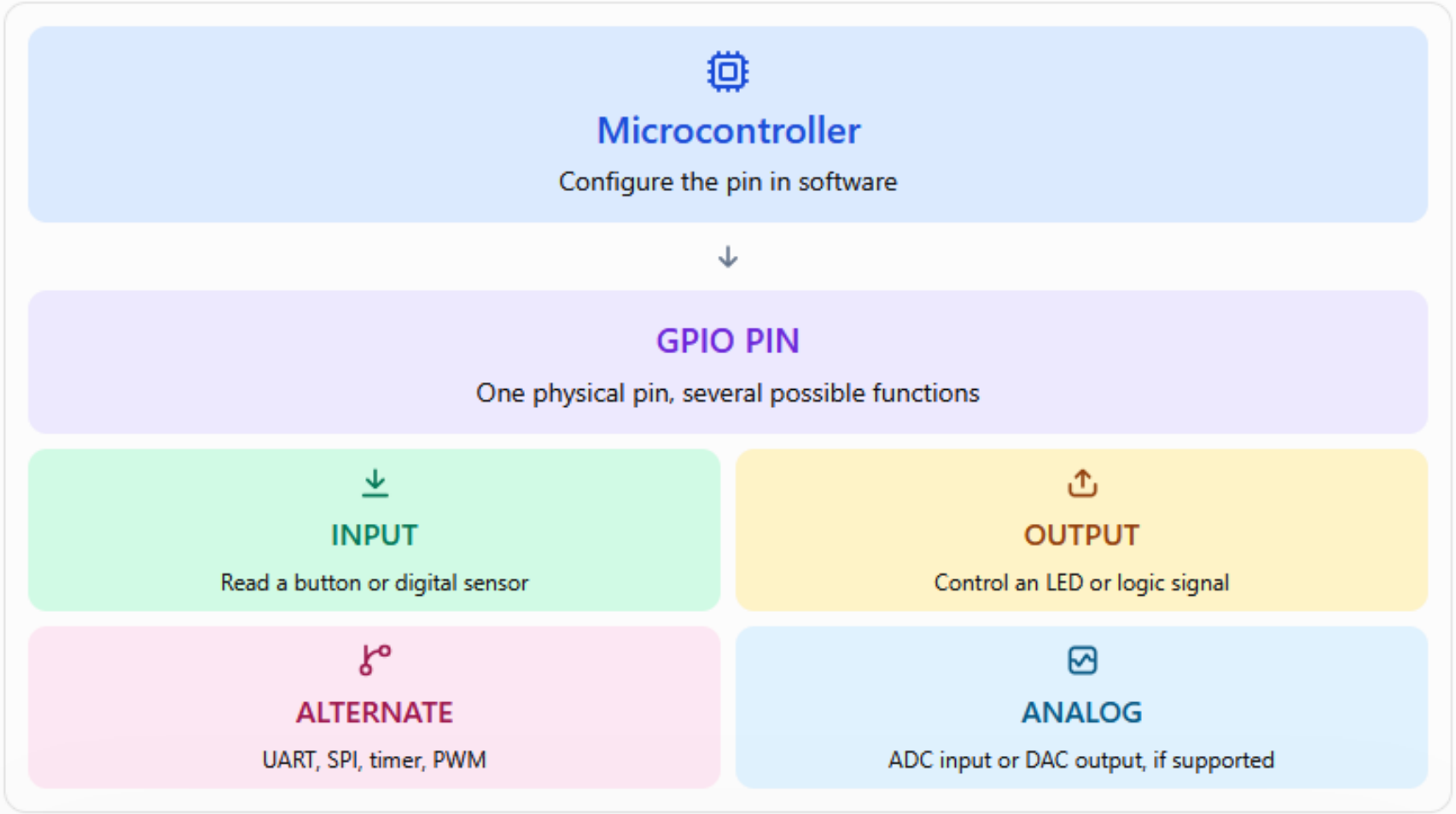
4. GPIO

One of the most important concepts for practical embedded programming: **GPIO**.

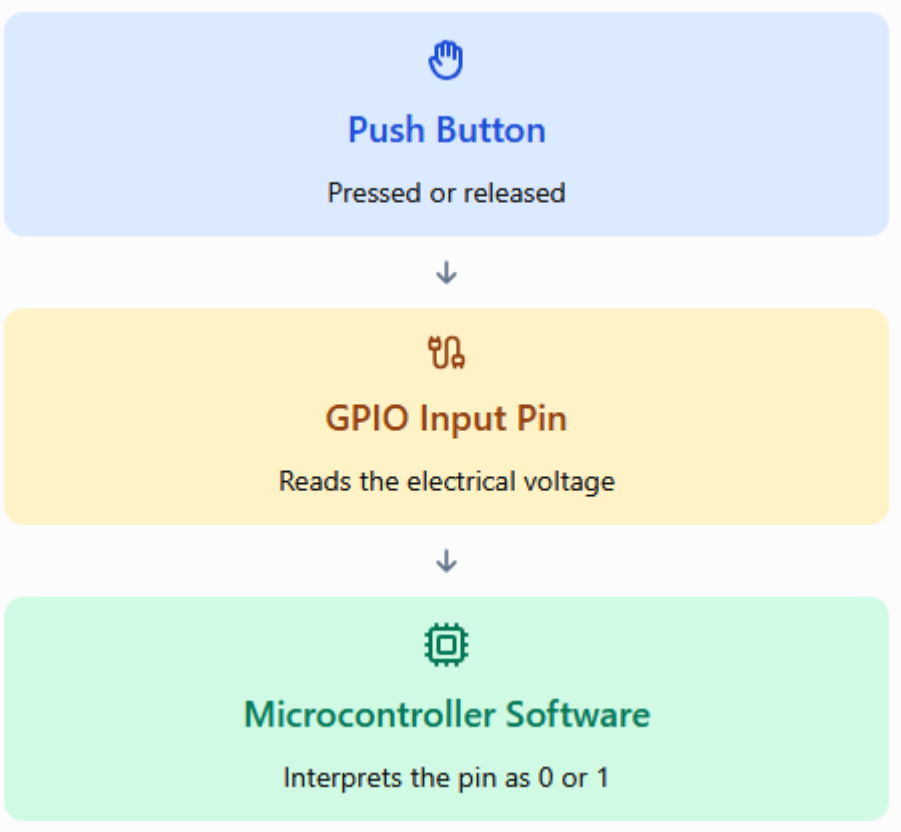
GPIO = General Purpose Input/Output

GPIO pins allow the microcontroller to communicate with external digital devices.

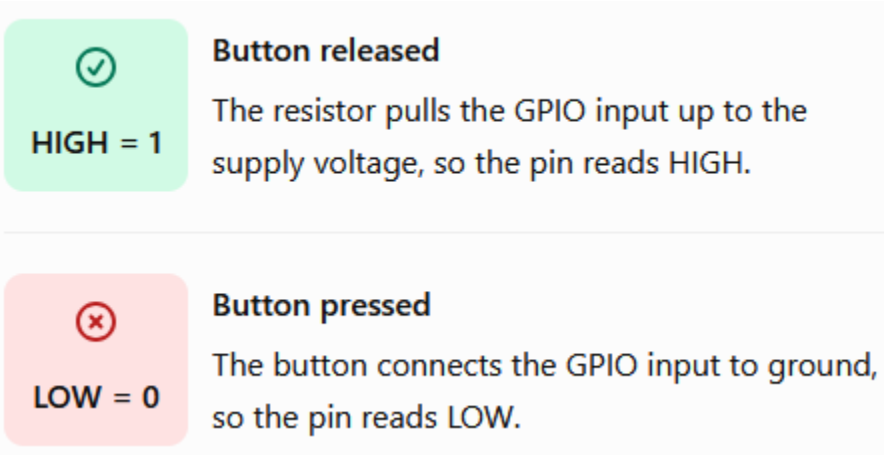
A GPIO can generally operate as ➔



GPIO Input



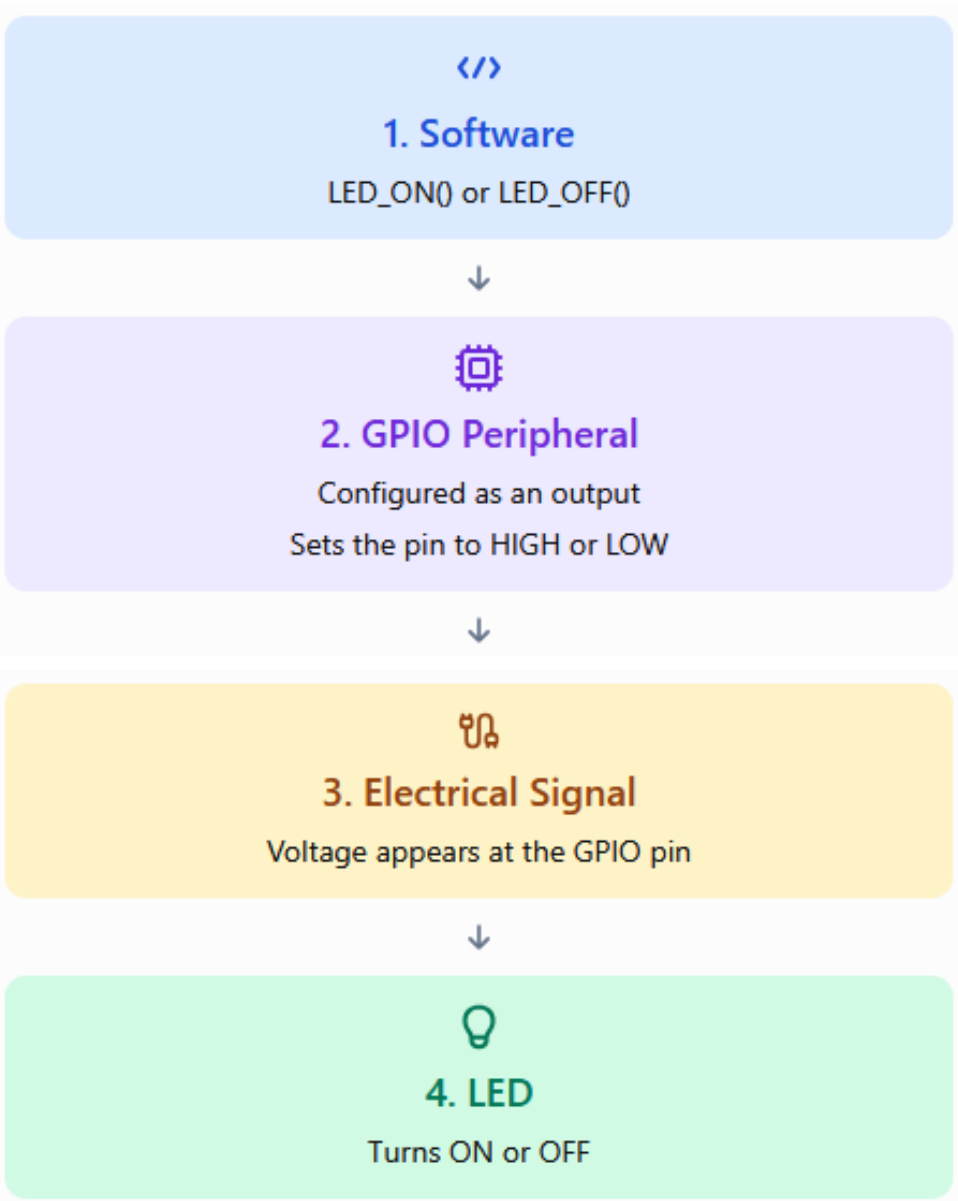
Possible values:



The button being pressed does not always mean HIGH.
In this pull-up circuit, pressing the button produces LOW.
This is called active-low logic.

4. GPIO

GPIO Output



Software controls the output:

HIGH (1)

LED_ON()

The GPIO outputs a HIGH voltage. In a typical active-HIGH circuit, current flows through the LED and its current-limiting resistor, so the LED lights up.

LOW (0)

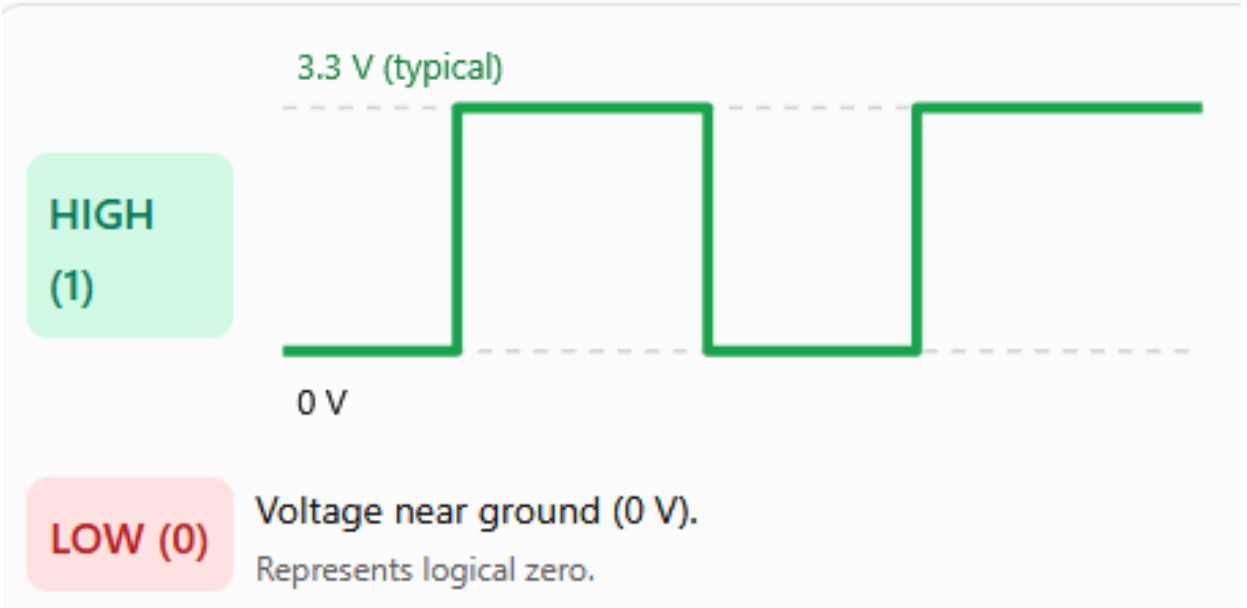
LED_OFF()

The GPIO outputs a LOW voltage. In the same circuit, current stops flowing through the LED, so it turns off.

5. DIGITAL SIGNAL

Digital Signal

A digital signal represents information using discrete logical states, usually LOW (0) and HIGH (1). In a microcontroller, these states correspond to different voltage levels.



For a typical 3.3 V MCU:

```
LOW  ≈ 0 V
HIGH ≈ 3.3 V
```

The exact electrical thresholds depend on the MCU and input type.

6. GPIO INPUT — BUTTON

Floating Input in a GPIO

Button closed (pressed)

VCC (3.3 V)

↓

Closed button

↓

GPIO input = HIGH (1)

Button open (released)

VCC (3.3 V)

↓

Open button — circuit disconnected

?

GPIO input = UNDEFINED

If there is no other connection to a defined voltage, the pin floats.
It might be read as HIGH or LOW unpredictably

by : Dr. L.ABADA

6. GPIO INPUT — BUTTON

Why is a floating input a problem?

A microcontroller expects a digital input to represent either logic 0 or logic 1.

A floating pin can:

- Pick up electrical noise from nearby circuits.
- Change between HIGH and LOW without the button being pressed.
- Cause false button detections and unpredictable program behavior.

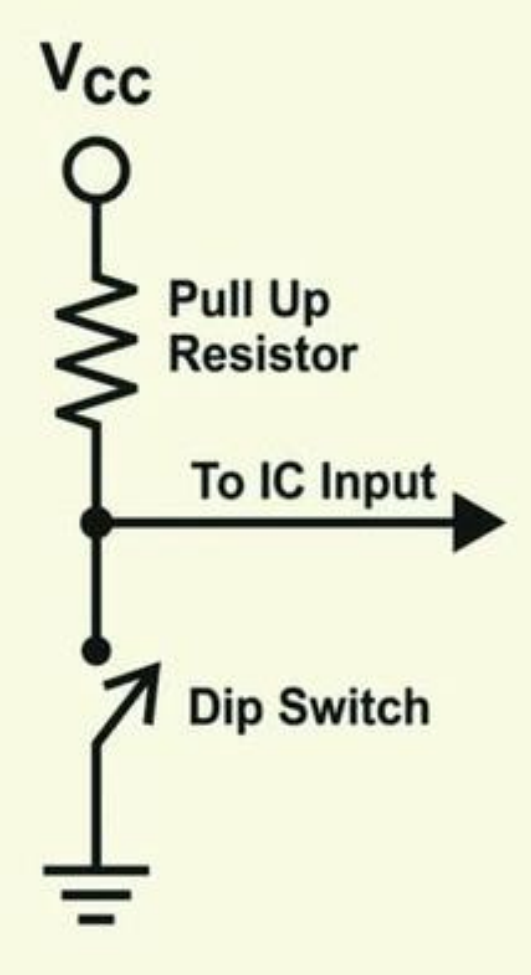
The solution is to use a pull-up resistor or a pull-down resistor.

7. PULL-UP

Pull-Up Resistor

The resistor pulls the GPIO toward HIGH

Typical configuration:



Button released ➔ `GPIO = HIGH`
Button pressed ➔ `GPIO = LOW`

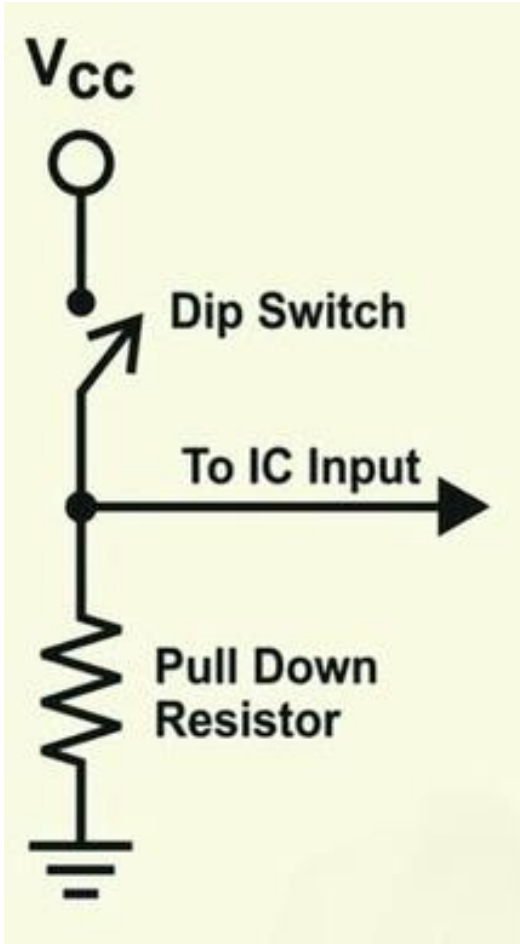
This is called an **active-low** button.

8. PULL-DOWN

Pull-Down Resistor

The resistor pulls the GPIO toward ground.

Typical configuration:



Button released ➔ `GPIO = LOW`
Button pressed ➔ `GPIO = HIGH`

8. PULL-DOWN

Internal Pull Resistors

Many microcontrollers provide internal:

- **Pull-up**
- **Pull-down**

This can simplify the circuit because we don't need an external resistor for every button.

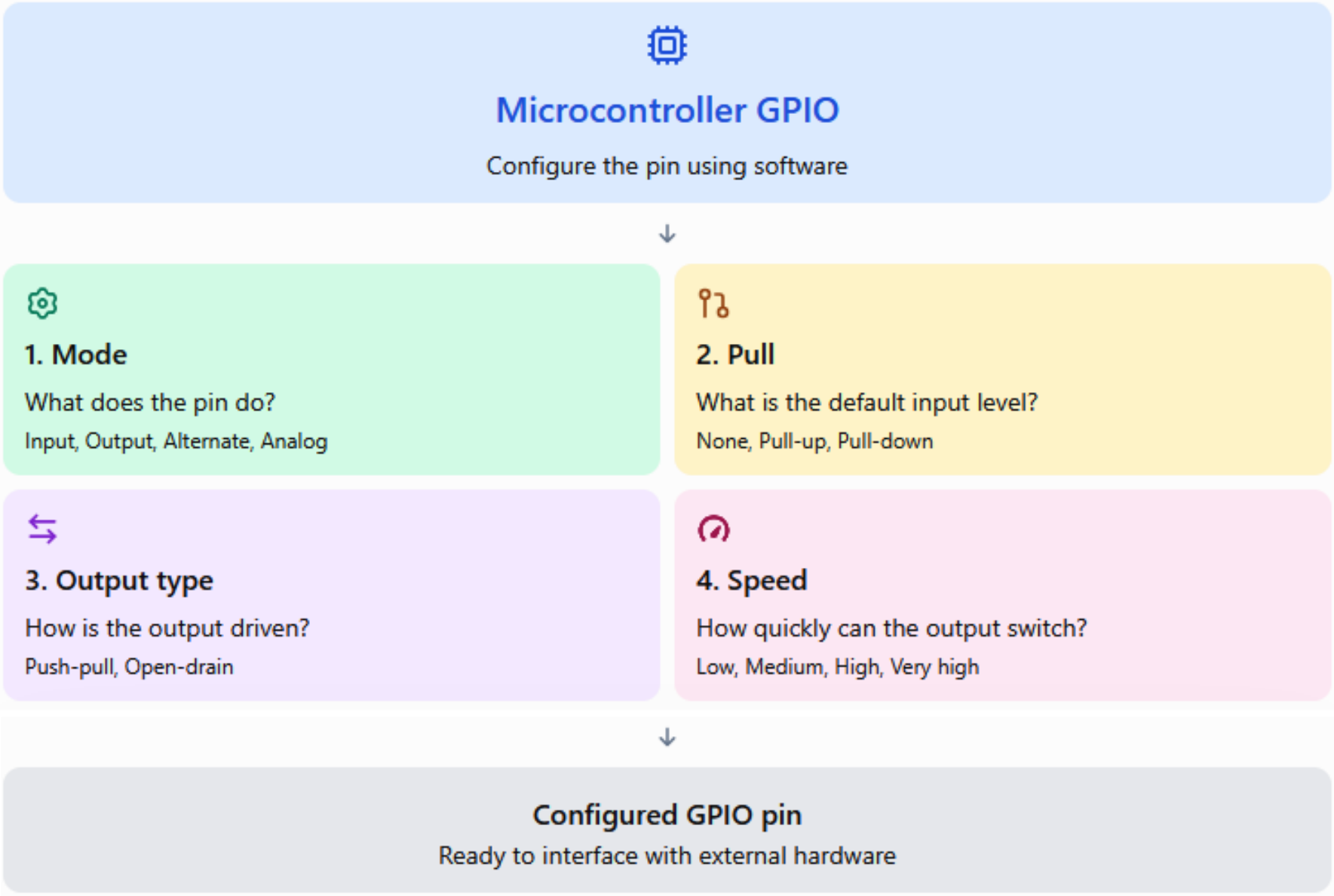
For STM32, GPIO configuration can typically specify:

- **`No Pull`**
- **`Pull-up`**
- **`Pull-down`**

The exact availability depends on the pin/MCU.

9. GPIO CONFIGURATION

GPIO Configuration Parameters : When configuring a GPIO, we may need to specify:



10. PUSH-PULL

Push-Pull Output

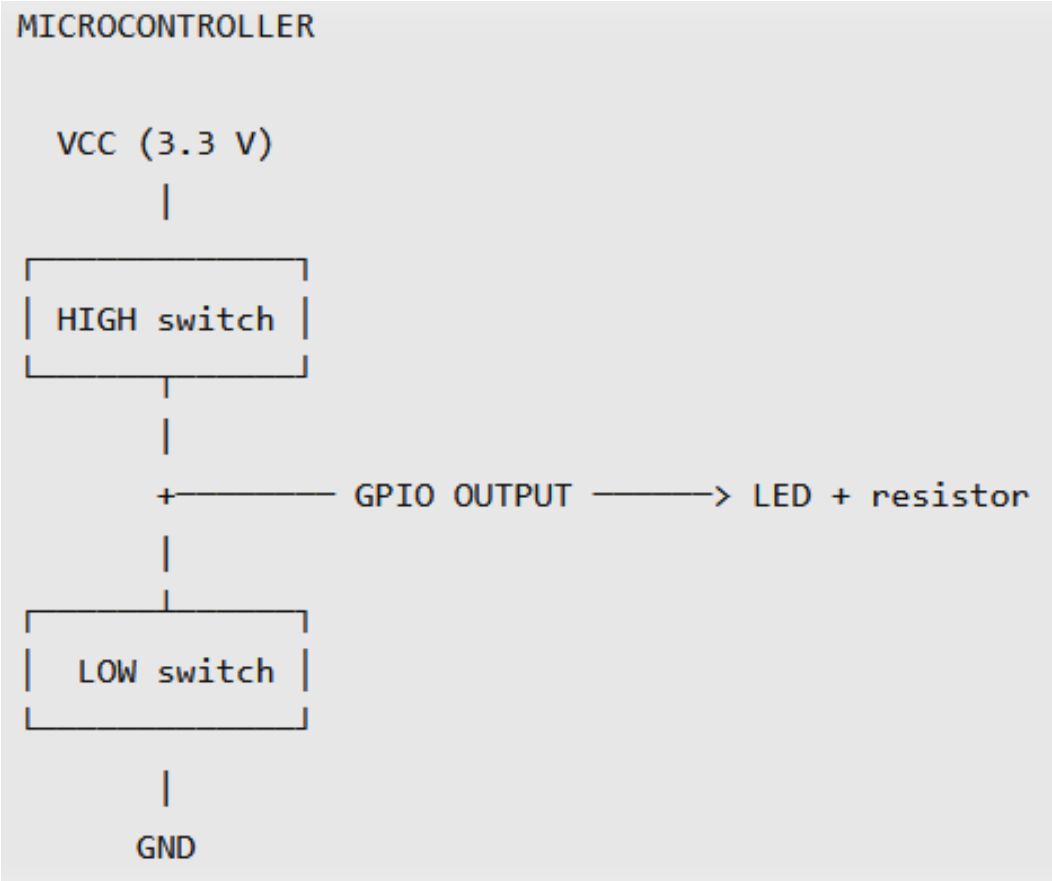
A push-pull output can actively drive:

HIGH

or

LOW

This is the common configuration for driving digital signals.

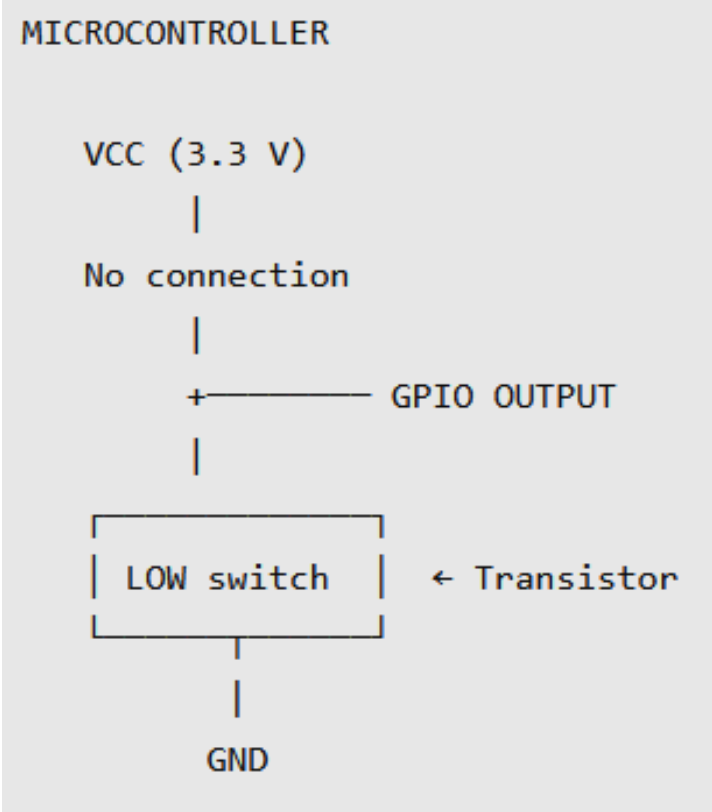


11. OPEN-DRAIN

Open-Drain Output
An open-drain output can actively pull the line low.
A pull-up is normally required.

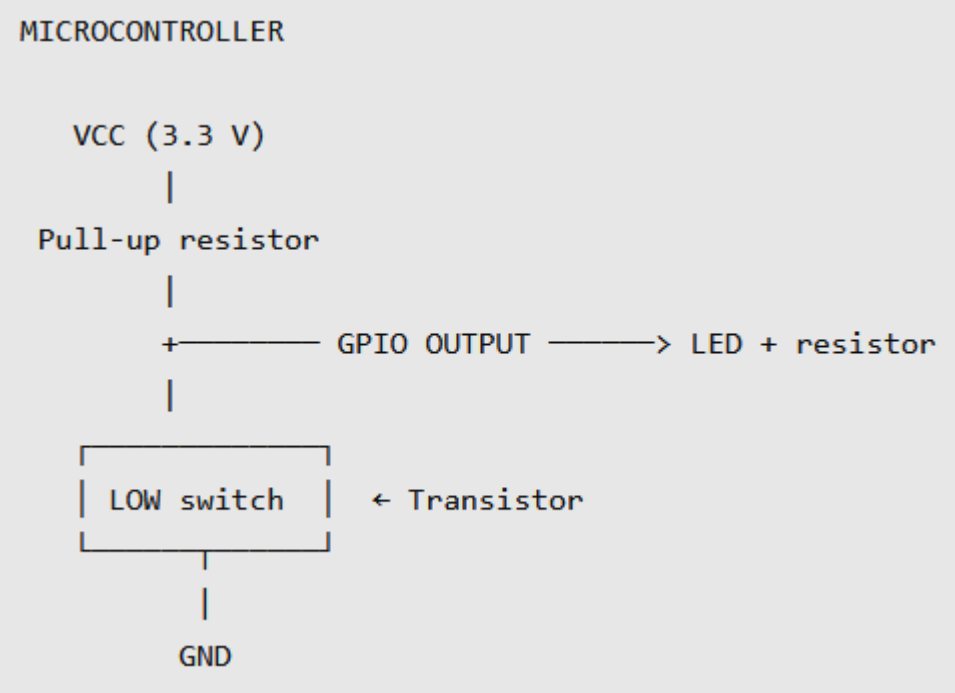
Used in applications such as:

- **I²C**
- **Shared buses**
- **Wired logic**



LOW switch ON: GPIO = **LOW (0)**, because the transistor connects it to GND.

LOW switch OFF: GPIO = High-impedance (**floating**), because neither VCC nor GND drives the output.



LOW switch ON: The transistor connects the output to GND → GPIO = **LOW (0)**.

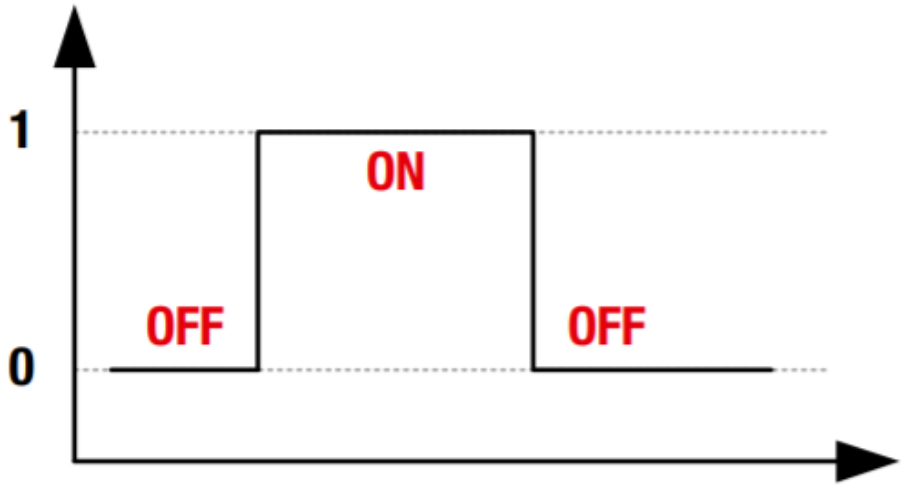
LOW switch OFF: The transistor disconnects from GND → the pull-up resistor brings GPIO **HIGH (1)**.

12. BUTTON DEBOUNCING

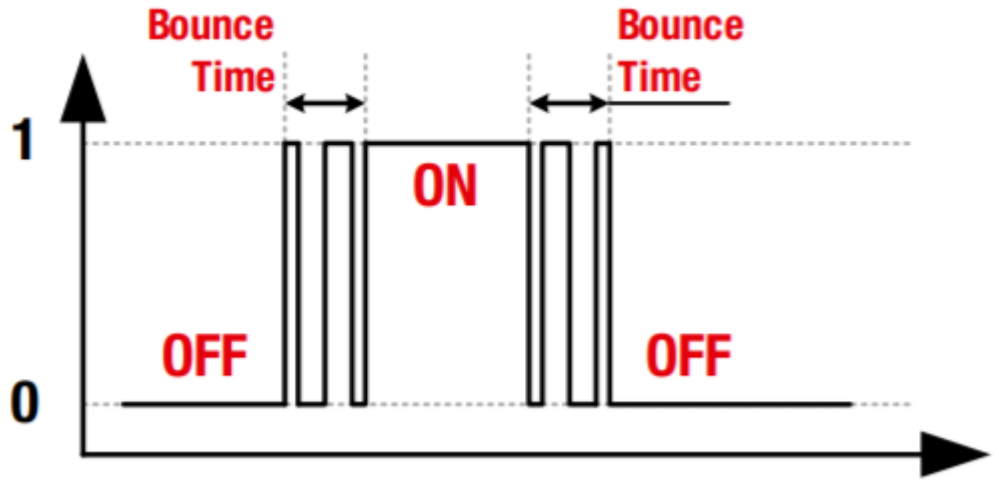
Mechanical Bounce

A mechanical button does not necessarily produce a clean transition.

Switch Contact Principle



Ideal situation of switching signal



This phenomenon is called bounce.

12. BUTTON DEBOUNCING

Why Debouncing?

Mechanical bounce is a problem because one physical button press can produce several electrical transitions, causing the microcontroller to execute an action multiple times.

Example: Incrementing a counter

Imagine that each time you press a button, the counter increases by 1.



The user presses the button once.

The GPIO signal bounces, and the software detects multiple transitions.

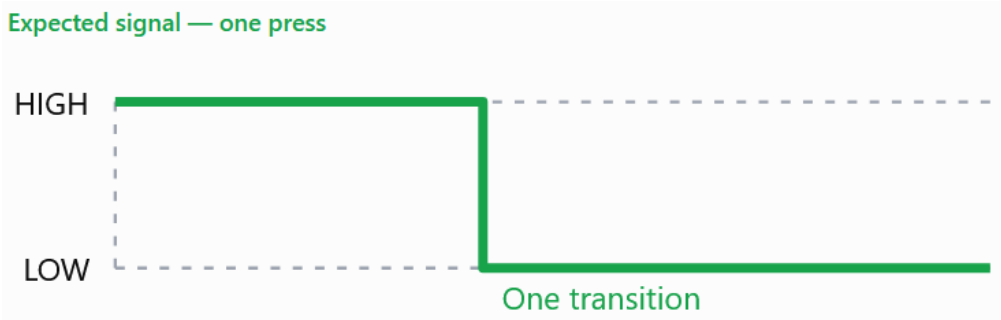
The counter increases five times instead of once.

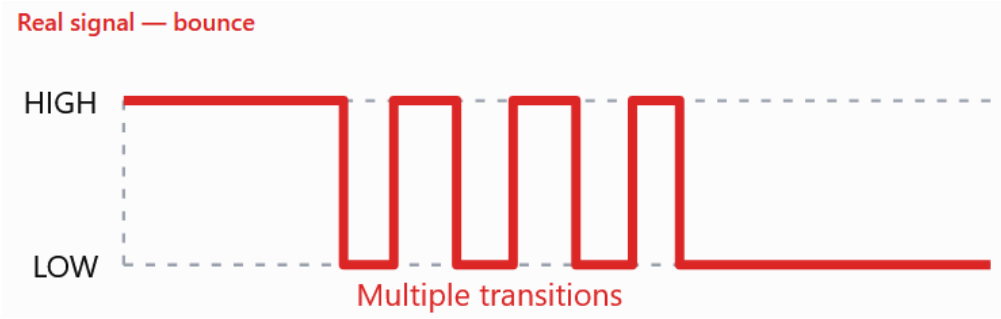
12. BUTTON DEBOUNCING

Software Debouncing

One simple approach:

```
Button changes
      ↓
Wait 10–50 ms
      ↓
Read again
      ↓
Same state?
      ↓
Accept the event
```





The exact delay should be selected according to the hardware/application.

12. BUTTON DEBOUNCING

Better Debouncing Concept

Instead of blindly waiting:

```
state change
     ↓
start timer
     ↓
wait for stable state
     ↓
confirm transition
```

This can be implemented without blocking the whole application.

This idea becomes important when we study multitasking and real-time systems.

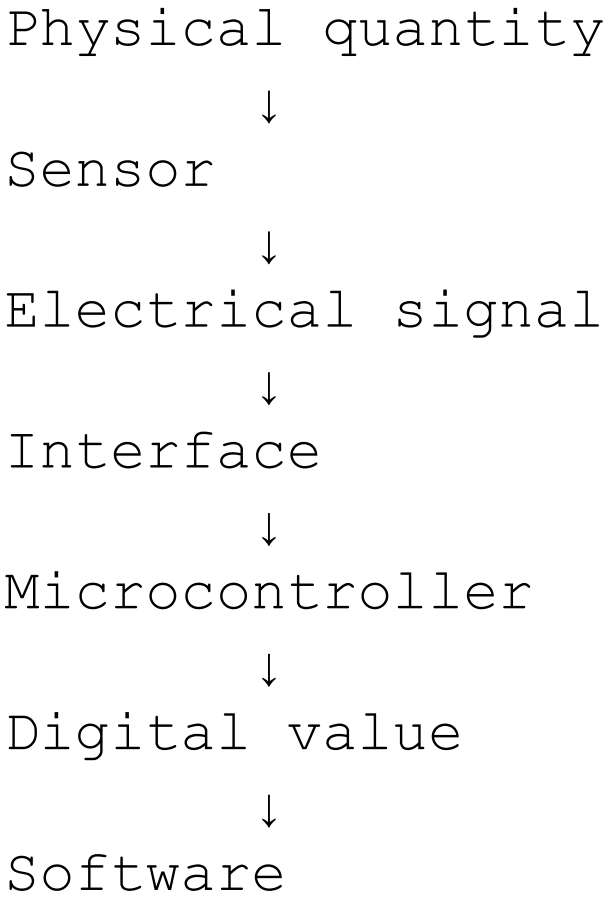
13. DATA ACQUISITION

What Is Data Acquisition?

Data acquisition means:

Obtaining measurements from the physical world and making them available to software.

Depending on the sensor, the interface can be GPIO, ADC, I^2C, SPI, UART, CAN, or another interface.



14. DIGITAL VS ANALOG

Digital Sensor

A digital sensor may provide: `0 / 1`

or communicate using:

- **I²C**
- **SPI**
- **UART**
- **CAN**
- **...**

Example:

A digital temperature sensor may measure the temperature internally and send the numerical result to the microcontroller through I²C.
In this case, the microcontroller does not necessarily need to perform an ADC conversion itself.

14. DIGITAL VS ANALOG

Analog Sensor

An analog sensor can produce a continuous voltage.

For example:

```
0 V       → minimum
1.2 V     → measured value
2.4 V     → measured value
3.3 V     → maximum
```

The CPU works with digital numbers

➔ we need a device that converts the analog voltage into a digital number.

This device is called an ADC.

ADC = Analog-to-Digital Converter

It converts:

Analog voltage

↓

ADC

↓

Digital number

For an N-bit ADC: Number of levels = 2^N

For example, a 12-bit ADC provides: $2^{12} = 4096$ possible digital codes.

ADC Example

For a 12-bit ADC and a 3.3 V reference:

`ADC = 0` ➔ `approximately 0 V`

`ADC = 4095` ➔ `approximately 3.3 V`

Approximate conversion : $ADC = \frac{V_{in}}{V_{ref}} \times 4095$

More bits generally give us finer quantization,

➔ **But, more resolution does not automatically mean more measurement accuracy.**

ADC Resolution

Suppose:

```
Vref = 3.3 V
12 bits ADC
```

The approximate voltage step is: 3.3/4096≈0.806 mV

Important

Resolution is not the same as accuracy.

Accuracy also depends on:

- Reference voltage
- Noise
- Sensor accuracy
- ADC characteristics
- PCB design
- Calibration

ADC?? For V_{in} = 1.7v, 1.8v, 2.2v

Digital-to-Analog Converter

A DAC performs the opposite conversion:

```
Digital value
      ↓
     DAC
      ↓
Analog voltage
```

Applications include:

- Audio
- Signal generation
- Control
- Analog outputs

Controlling an LED

The simplest actuator:

GPIO

↓

LED

Software:

```
LED_ON();
```

The MCU changes the GPIO output.

Controlling a Motor

A motor generally cannot be driven directly from a microcontroller GPIO.

Why?

Because the motor may require:

- Higher current
- Higher voltage
- Electrical protection

Typical architecture:

- MCU
- ↓
- **Control signal**
- ↓
- **Driver / transistor / ESC**
- ↓
- Motor

GPIO can only provide a limited amount of current and voltage.

Motor may require much more current that could damage the microcontroller.

18. PWM

PWM = Pulse Width Modulation

Is a technique used by a microcontroller to control the power delivered to a device by switching a digital signal between HIGH and LOW very quickly.

It is commonly used to control LED brightness, motor speed, and servo position.

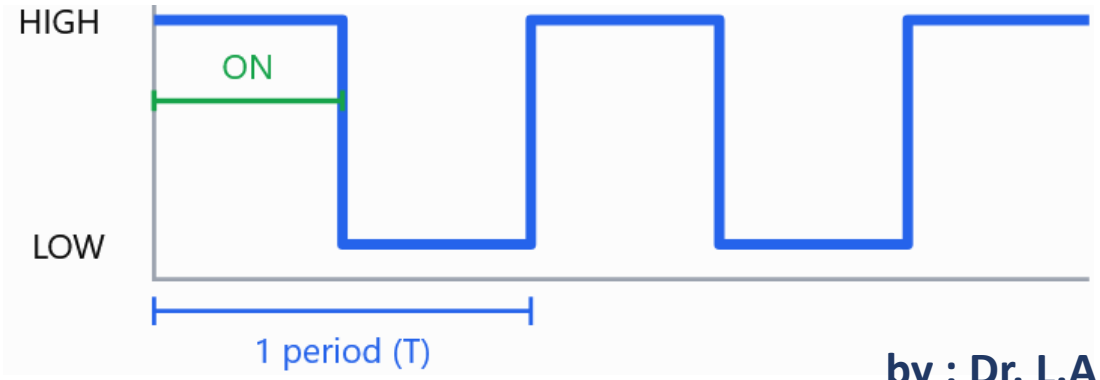
How does PWM work?

A PWM signal repeatedly switches between two states:

- **HIGH: The signal is ON.**
- **LOW: The signal is OFF.**

Instead of keeping the signal HIGH continuously, we control how long it stays HIGH during each cycle. The percentage of time spent HIGH is called the **duty cycle**.

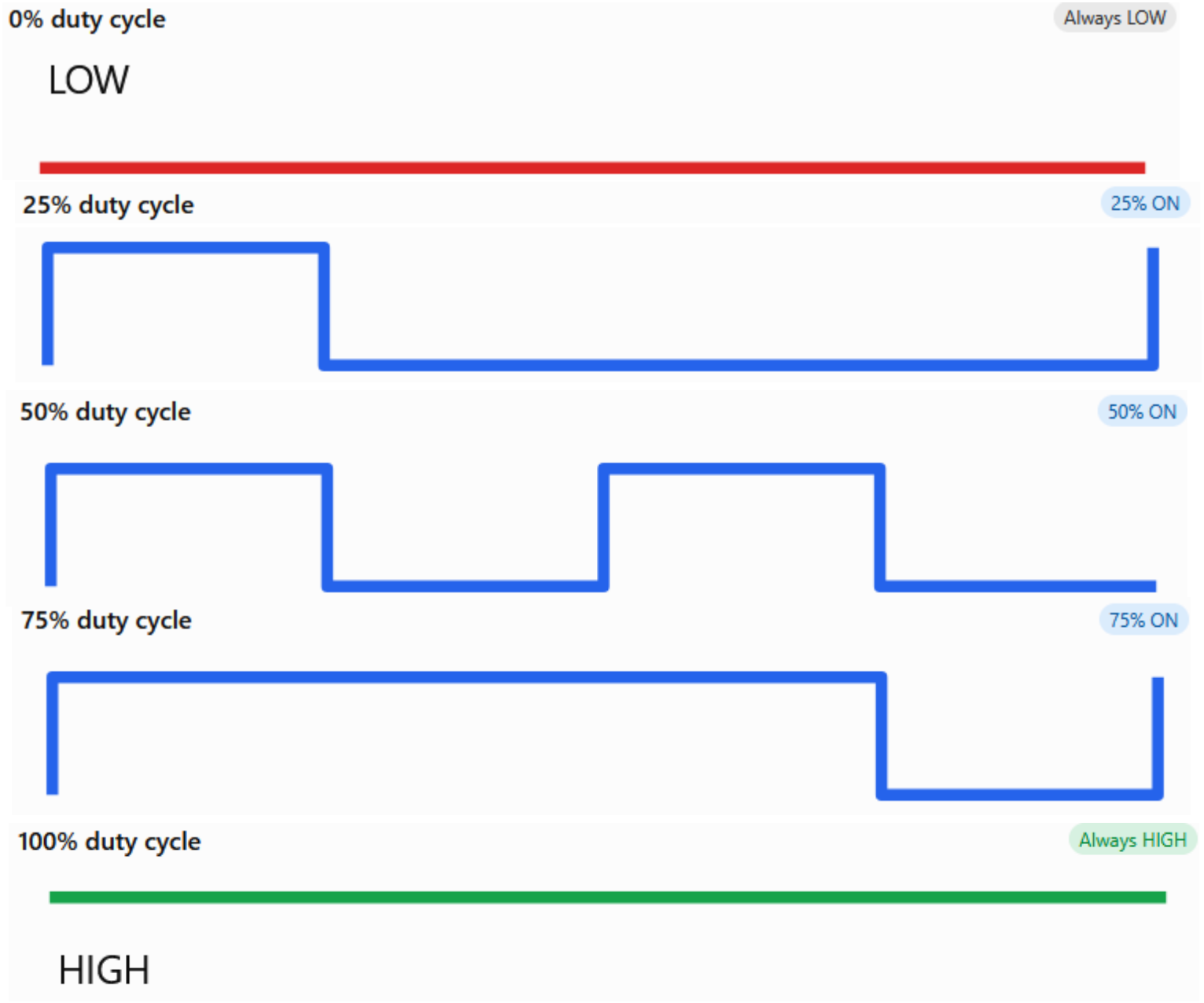
Example: PWM signal with a 50% duty cycle



18. PWM

Duty Cycle

For example:



18. PWM

PWM Applications

PWM can be used for:

- LED brightness
- Motor control
- Servo control
- Power control
- Heating control

Important

PWM does not mean that the GPIO produces an intermediate analog voltage.
It rapidly switches between digital levels.



25% duty cycle
The LED is ON for a small part of each period.
Low average brightness



50% duty cycle
The LED is ON for half of each period.
Medium average brightness



90% duty cycle
The LED is ON for most of each period.
High average brightness

Why Timers?

A timer is a hardware peripheral inside a microcontroller that allows us to measure time, generate periodic events, and control when certain operations happen.

Examples:

- Read a sensor every 10 ms
- Generate PWM
- Measure pulse duration
- Generate periodic events
- Schedule tasks
- Implement timeouts

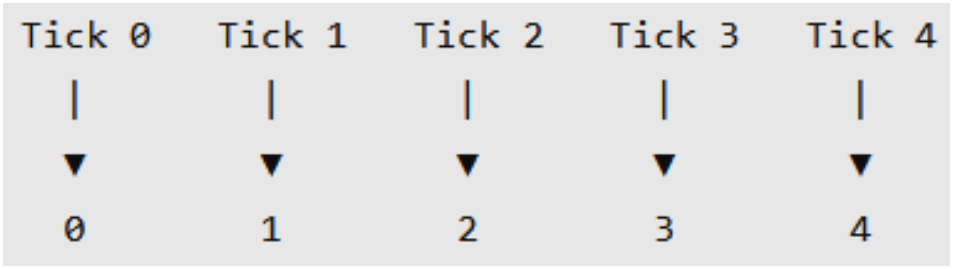
How does a timer measure time?

Imagine that a timer receives a clock running at 1 MHz.
That means it receives 1,000,000 clock ticks per second.

$$T_{\text{tick}} = \frac{1}{f_{\text{clock}}}$$

$$T_{\text{tick}} = \frac{1}{1,000,000} = 1\,\mu s$$

Each timer tick represents one microsecond.



20. TIMER FREQUENCY

Basic Timer Calculation

Suppose a timer receives a clock:

ftimer=1MHz

Then:

`1 clock = 1 µs`

If the timer counts 1000 ticks:

1000×1µs=1ms

Therefore the event occurs every: **1 ms**

20. TIMER FREQUENCY

The prescaler (PSC)

The prescaler divides the timer input clock to produce a slower counter clock.
For STM32 timers, the division factor is commonly:

$$f_{\text{counter}} = \frac{f_{\text{timer}}}{PSC + 1}$$

For example, if the timer clock is 1 MHz and PSC = 9:

$$f_{\text{counter}} = \frac{1,000,000}{9 + 1} = 100,000 \text{ Hz}$$

Each counter tick now takes 10 microseconds instead of 1 microsecond.

20. TIMER FREQUENCY

The counter period (ARR)

The auto-reload register (ARR) determines the counter's upper limit. In the common up-counting mode, the counter runs from 0 through ARR, then rolls over and generates an update event.

The period is therefore:

$$T = \frac{(PSC + 1)(ARR + 1)}{f_{\text{timer}}}$$

The +1 terms matter because the prescaler and counter count include zero.

Example A: Generate an event every 1 ms

Given:

- Timer clock = 1 MHz
- PSC = 0 (no division)
- ARR = 999

Then:

$$T = \frac{(0 + 1)(999 + 1)}{1,000,000} = \frac{1000}{1,000,000} = 1 \text{ ms}$$

Blocking Delay

Example:

```
HAL_GPIO_TogglePin(...);
HAL_Delay(1000);
```

During the delay, the current execution flow is blocked.

```
Toggle
  ↓
WAIT 1 second
  ↓
Continue
```

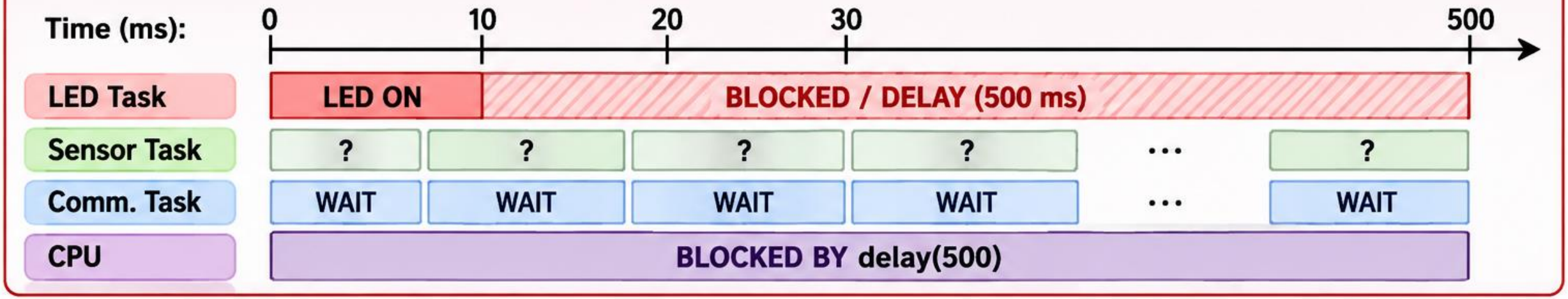
21. BLOCKING TIMING

Problem With Blocking

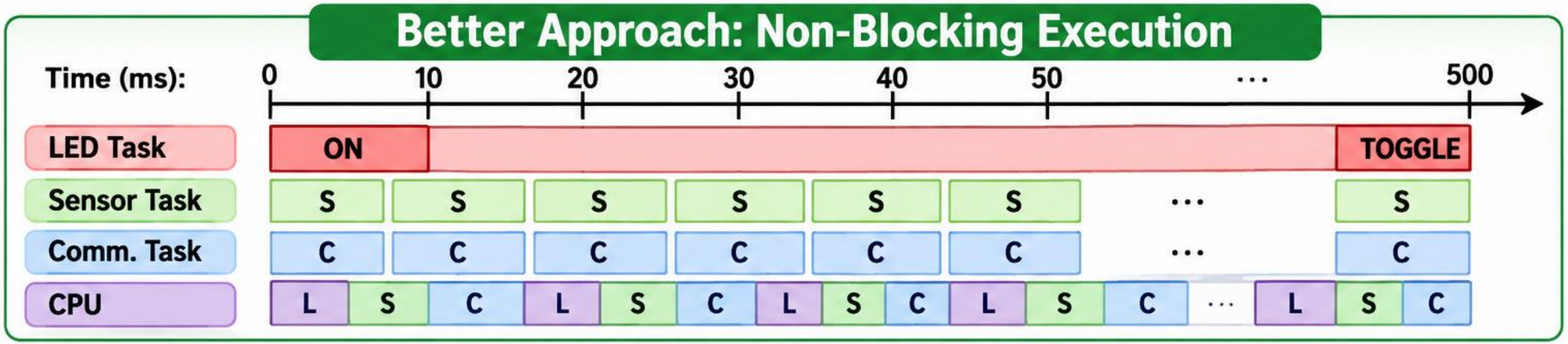
Requirements:

- LED task : every 500 ms
- Sensor task : every 10 ms
- Communication : continuously

⚠ Using long blocking delays can make the software difficult to design.



21. BLOCKING TIMING



LED: Toggle every 500 ms
Sensor: Read every 10 ms
Communication: Process continuously

Techniques Used

- Timers
- Interrupts
- Event-driven programming
- Multitasking

22. NON-BLOCKING TIMING

Non-Blocking Concept

Instead of: `delay(1000);`

we can ask: `Has 1000 ms elapsed?`

Example:

```
if (HAL_GetTick() - last >= 1000)
{
    last = HAL_GetTick();
    toggle_led();
}
```

The program can continue doing other work.

23. EMBEDDED CONTROL LOOP

Basic Control Loop

A very common structure is:

```
while (1)
{
    read_sensor();

    process_data();

    control_actuator();

    communicate();

    update_status();
}
```

This is a very simple embedded architecture.

Later, we will improve it using:

- Interrupts
- Timers
- Tasks
- RTOS

24. PRACTICAL EXAMPLE

Automatic Fan

Imagine a temperature-controlled fan.

Requirement:

If temperature > 30°C, turn the fan ON.

Architecture:

Temperature sensor

↓

MCU

↓

Decision logic

↓

Motor driver

↓

Fan

24. PRACTICAL EXAMPLE

Software Logic

```
Read temperature
        ↓
temperature > 30°C ?
      /       \
    YES        NO
     ↓          ↓
Fan ON      Fan OFF
```

Example pseudocode:

```
temperature =
read_temperature();

if (temperature > 30)
    fan_on();
else
    fan_off();
```

Hysteresis

A simple threshold can cause repeated switching:

```
29.9°C → OFF
30.1°C → ON
29.9°C → OFF
30.1°C → ON
...
```

Use hysteresis:

```
Temperature > 30°C → ON
Temperature < 28°C → OFF
```

Between 28°C and 30°C:

Keep the previous state.

This is a very important practical control concept.

26. SOFTWARE / HARDWARE INTERFACE

Complete Chain

Students should now see the complete chain:

```
Physical quantity
        ↓
     Sensor
        ↓
 Electrical signal
        ↓
 GPIO / ADC / Bus
        ↓
 Microcontroller
        ↓
 Software
        ↓
 Decision
        ↓
 GPIO / PWM / Bus
        ↓
 Driver
        ↓
 Actuator
        ↓
Physical action
```

This is the core of embedded software development.

Slide 49 — Question 1

Why should an input not be left floating?

Expected answer:

Because its logic level may become undefined and sensitive to electrical noise.

Question 2

What is the difference between:

Sensor

and

Actuator?

```
Sensor:
physical → electrical/data

Actuator:
electrical/data → physical action
```

Question 3

Why can't a GPIO directly drive a large motor?

Because the GPIO cannot normally provide the required voltage/current and may be damaged.
A suitable driver is required.

Question 4

What is PWM?

A technique that controls the average delivered power or another effective quantity by changing the duty cycle of a periodic digital signal.

Question 5

Why is non-blocking timing useful?

It allows the processor/software to perform other activities while waiting for a future event.